Summarize

Chapter 2. Discerning Coupling in Software Architecture

Wednesday, November 3, 13:00



Logan, the lead architect for Penultimate Electronics, interrupted a small group of architects in the cafeteria, discussing distributed architectures. "Austen, are you wearing a cast *again*?"

"No, it's just a splint," replied Austen. "I sprained my wrist playing extreme disc golf over the weekend—it's almost healed."

"What is...never mind. What is this impassioned conversation I barged in on?"

"Why wouldn't someone always choose the *saga pattern* in microservices to wire together transactions?" asked Austen. "That way, architects can make the services as small as they want."

"But don't you have to use *orchestration* with sagas?" asked Addison. "What about times when we need asynchronous communication? And, how complex will the transactions get? If we break things down too much, can we really guarantee data fidelity?"

"You know," said Austen, "if we use an *enterprise service bus*, we can get it to manage most of that stuff for us."

"I thought no one used ESBs anymore—shouldn't we use Kafka for stuff like that?"

"They aren't even the same thing!" said Austen.

Logan interrupted the increasingly heated conversation. "It is an apples-to-oranges comparison, but none of these tools or approaches is a silver bullet. Distributed architectures like microservices are difficult, especially if architects cannot untangle all the forces at play. What we need is an approach or framework that helps us figure out the hard problems in our architecture."

"Well," said Addison, "whatever we do, it has to be as decoupled as possible—everything I've read says that architects should embrace decoupling as much as possible."

"If you follow that advice," said Logan, "Everything will be so decoupled that nothing can communicate with anything else—it's hard to build software that way! Like a lot of things, coupling isn't inherently bad; architects just have to know how to apply it appropriately. In fact, I remember a famous quote about that from a philosopher...."

> *All things are poison, and nothing is without poison; the dosage alone makes it so a thing is not a poison.*
>
> —Paracelsus

One of the most difficult tasks an architect will face is untangling the various forces and trade-offs at play in distributed architectures. People who provide advice constantly extol the benefits of "loosely coupled" systems, but how can architects design systems where nothing connects to anything else? Architects design fine-grained microservices to achieve decoupling, but then orchestration, transactionality, and asynchronicity become huge problems. Generic advice says "decouple," but provides no guidelines for *how* to achieve that goal while still constructing useful systems.

Architects struggle with granularity and communication decisions because there are no clear universal guides for making decisions—no best practices exist that can apply to real-world complex systems. Until now, architects lacked the correct perspective and terminology to allow a careful analysis that could determine the best (or least worst) set of trade-offs on a case-by-case basis.

Why have architects struggled with decisions in distributed architectures? After all, we've been building distributed systems since the last century, using many of the same mechanisms (message queues, events, and so on). Why has the complexity ramped up so much with microservices?

The answer lies with the fundamental philosophy of microservices, inspired by the idea of a *bounded context*. Building services that model bounded contexts required a subtle but important change to the way architects designed distributed systems because now transactionality is a first-class architectural concern. In many of the distributed systems architects designed prior to microservices, event handlers typically connected to a single relational database, allowing it to handle details such as integrity and transactions. Moving the database within the service boundary moves data concerns into architecture concerns.

As we've said before, *"Software architecture" is the stuff you can't Google answers for*. A skill that modern architects must build is the ability to do trade-off analysis. While several frameworks have existed for decades (such as Architecture Trade-off Analysis Method, or ATAM), they lack focus on real problems architects face on a daily basis.

This book focuses on how architects can perform trade-off analysis for any number of scenarios unique to their situation. As in many things in architecture, the advice is simple; the hard parts lie in the details, particularly how difficult parts become entangled, making it difficult to see and understand the individual parts, as illustrated in Figure 2-1.

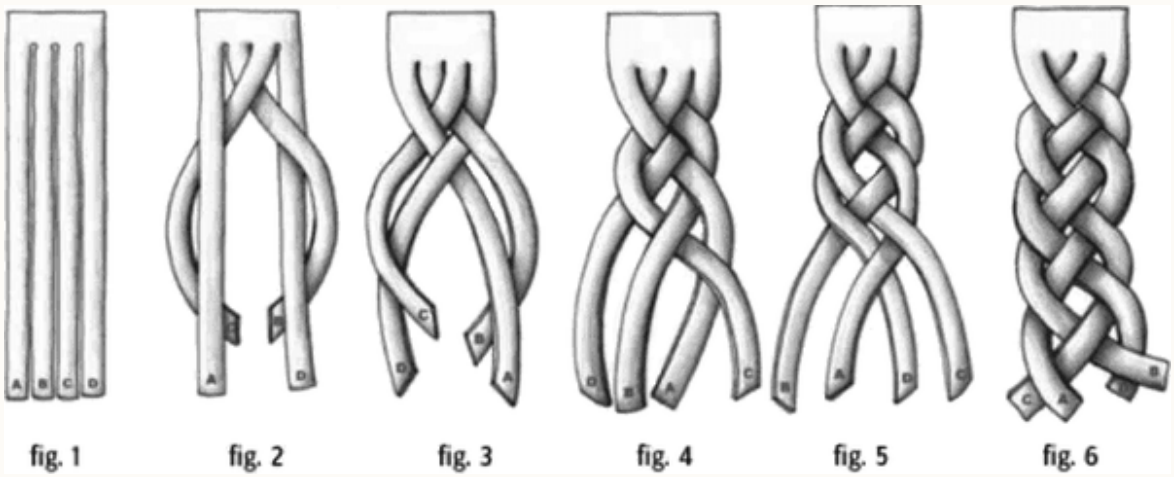


Figure 2-1. A braid entangles hair, making the individual strands hard to identify

When architects look at entangled problems, they struggle with performing trade-off analysis because of the difficulties separating the concerns, so that they may consider them independently. Thus, the first step in trade-off analysis is untangle the dimensions of the problem, analyzing what parts are coupled to one another and what impact that coupling has on change. For this purpose, we use the simplest definition of the word *coupling*:

Coupling

> Two parts of a software system are coupled if a change in one might cause a change in the other.

Often, software architecture creates multidimensional problems, where multiple forces all interact in interdependent ways. To analyze trade-offs, an architect must first determine what forces need to trade off with each other.

Thus, here's our advice for modern trade-off analysis in software architecture:

1. Find what parts are entangled together.
2. Analyze how they are coupled to one another.
3. Assess trade-offs by determining the impact of change on interdependent systems.

While the steps are simple, the hard parts lurk in the details. Thus, to illustrate this framework in practice, we take one of the most difficult (and probably the closest to generic) problems in distributed architectures, which is related to microservices:

How do architects determine the size and communication styles for microservices?

Determining the proper size for microservices seems a pervasive problem—too-small services create transactional and orchestration issues, and too-large services create scale and distribution issues.

To that end, the remainder of this book untangles the many aspects to consider when answering the preceding question. We provide new terminology to differentiate similar but distinct patterns and show practical examples of applying these and other patterns.

However, the overarching goal of this book is to provide you with example-driven techniques to learn how to construct your own trade-off analysis for the unique problems within your realm. We start with our first great untangling of forces in distributed architectures: defining architecture quantum along with the two types of coupling, static and dynamic.

Architecture (Quantum | Quanta)

The term *quantum* is, of course, used heavily in the field of physics known as *quantum mechanics*. However, the authors chose the word for the same reasons physicists did. *Quantum* originated from the Latin word *quantus*, meaning "how great" or "how many." Before physics co-opted it, the legal profession used it to represent the "required or allowed amount" (for example, in damages paid). The term also appears in the mathematics field of topology, concerning the properties of families of shapes. Because of its Latin roots, the singular is *quantum*, and the plural is *quanta*, similar to the datum/data symmetry.

An architecture quantum measures several aspects of both topology and behavior in software architecture related to how parts connect and communicate with one another:

Architecture quantum

An architecture quantum is an independently deployable artifact with high functional cohesion, high static coupling, and synchronous dynamic coupling. A common example of an architecture quantum is a well-formed microservice within a workflow.

Static coupling

Represents how static dependencies resolve within the architecture via contracts. These dependencies include operating system, frameworks, and/or libraries delivered

via transitive dependency management, and any other operational requirement to allow the quantum to operate.

Dynamic coupling

Represents how quanta communicate at runtime, either synchronously or asynchronously. Thus, fitness functions for these characteristics must be *continuous*, typically utilizing monitors.

Even though both static and dynamic coupling seem similar, architects must distinguish two important differences. An easy way to think about the difference is that *static coupling* describes how services are *wired* together, whereas *dynamic coupling* describes how services *call* one another at runtime. For example, in a microservices architecture, a service must contain dependent components such as a database, representing static coupling—the service isn't operational without the necessary data. That service may call other services during the course of a workflow, which represents dynamic coupling. Neither service requires the other to be present to function, except for this runtime workflow. Thus, static coupling analyzes operational dependencies, and dynamic coupling analyzes communication dependencies.

These definitions include important characteristics; let's cover each in detail as they inform most of the examples in the book.

Independently Deployable

Independently deployable implies several aspects of an architecture quantum—each quantum represents a separate deployable unit within a particular architecture. Thus, a monolithic architecture—one that is deployed as a single unit—is by definition a single architecture quantum. Within a distributed architecture such as microservices, developers tend toward the ability to deploy services independently, often in a highly automated way. Thus, from an independently deployable standpoint, a service within a microservices architecture represents an architecture quantum (contingent on coupling—as discussed next).

Making each architecture quantum represent a deployable asset within the architecture serves several useful purposes. First, the boundary represented by an architecture quantum serves as a useful common language among architects, developers, and operations. Each understands the common scope under question: architects understand the coupling characteristics, developers understand the scope of behavior, and the operations team understands the deployable characteristics.

Second, the architecture quantum represents one of the forces (static coupling) architects must consider when striving for proper granularity of services within a distributed architec-

ture. Often, in microservices architectures, developers face the difficult question of what service granularity offers the optimum set of trade-offs. Some of those trade-offs revolve around deployability: what release cadence does this service require, what other services might be affected, what engineering practices are involved, and so on. Architects benefit from a firm understanding of exactly where deployment boundaries lie in distributed architectures. We discuss service granularity and its attendant trade-offs in Chapter 7.

Third, *independent deployability* forces the architecture quantum to include common coupling points such as databases. Most discussions about architecture conveniently ignore issues such as databases and user interfaces, but real-world systems must commonly deal with those problems. Thus, any system that uses a shared database fails the architecture quantum criteria for independent deployment unless the database deployment is in lockstep with the application. Many distributed systems that would otherwise qualify for multiple quanta fail the independently deployable part if they share a common database that has its own deployment cadence. Thus, merely considering the deployment boundaries doesn't solely provide a useful measure. Architects should also consider the second criteria for an architecture quantum, high functional cohesion, to limit the architecture quantum to a useful scope.

High Functional Cohesion

High functional cohesion refers structurally to the proximity of related elements: classes, components, services, and so on. Throughout history, computer scientists defined a variety of cohesion types, scoped in this case to the generic *module*, which may be represented as *classes* or *components*, depending on platform. From a domain standpoint, the technical definition of *high functional cohesion* overlaps with the goals of the *bounded context* in domain-driven design: behavior and data that implements a particular domain workflow.

From a purely independent deployability standpoint, a giant monolithic architecture qualifies as an architecture quantum. However, it almost certainly isn't highly functionally cohesive, but rather includes the functionality of the entire system. The larger the monolith, the less likely it is singularly functionally cohesive.

Ideally, in a microservices architecture, each service models a single domain or workflow, and therefore exhibits high functional cohesion. Cohesion in this context isn't about how services interact to perform work, but rather how independent and coupled one service is to another service.

High Static Coupling

High static coupling implies that the elements inside the architecture quantum are tightly wired together, which is really an aspect of contracts. Architects recognize things like REST or SOAP as contract formats, but method signatures and operational dependencies (via coupling points such as IP addresses or URLs) also represent contracts. Thus, contracts are an *architecture hard part*; we cover coupling issues involving all types of contracts, including how to choose appropriate ones, in Chapter 13.

An architecture quantum is, in part, a measure of static coupling, and the measure is quite simple for most architecture topologies. For example, the following diagrams show the architecture styles featured in *Fundamentals of Software Architecture*, with the architecture quantum static coupling illustrated.

Any of the monolithic architecture styles will necessarily have a quantum of one, as illustrated in Figure 2-2.

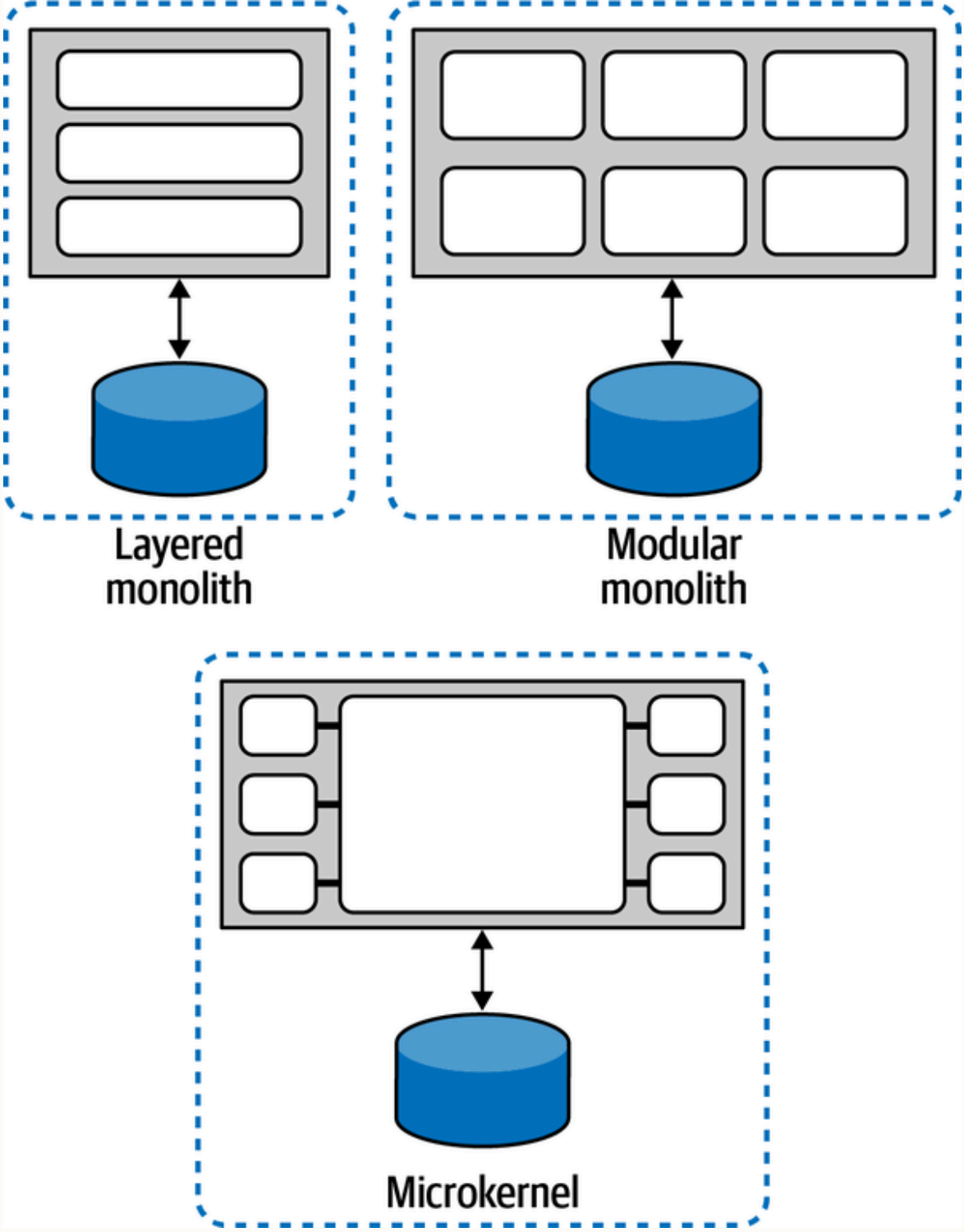


Figure 2-2. Monolithic architectures always have a quantum of one

As you can see, any architecture that deploys as a single unit and utilizes a single database will always have a single quantum. The architecture quantum measure of static coupling includes the database, and a system that relies on a single database cannot have more than a single quantum. Thus, the static coupling measure of an architecture quantum helps identify coupling points in architecture, not just within the software components under development. Most monolithic architectures contain a single coupling point (typically, a database) that makes its quantum measure one.

Distributed architectures often feature decoupling at the component level; consider the next set of architecture styles, starting with the service-based architecture shown in Figure 2-3.

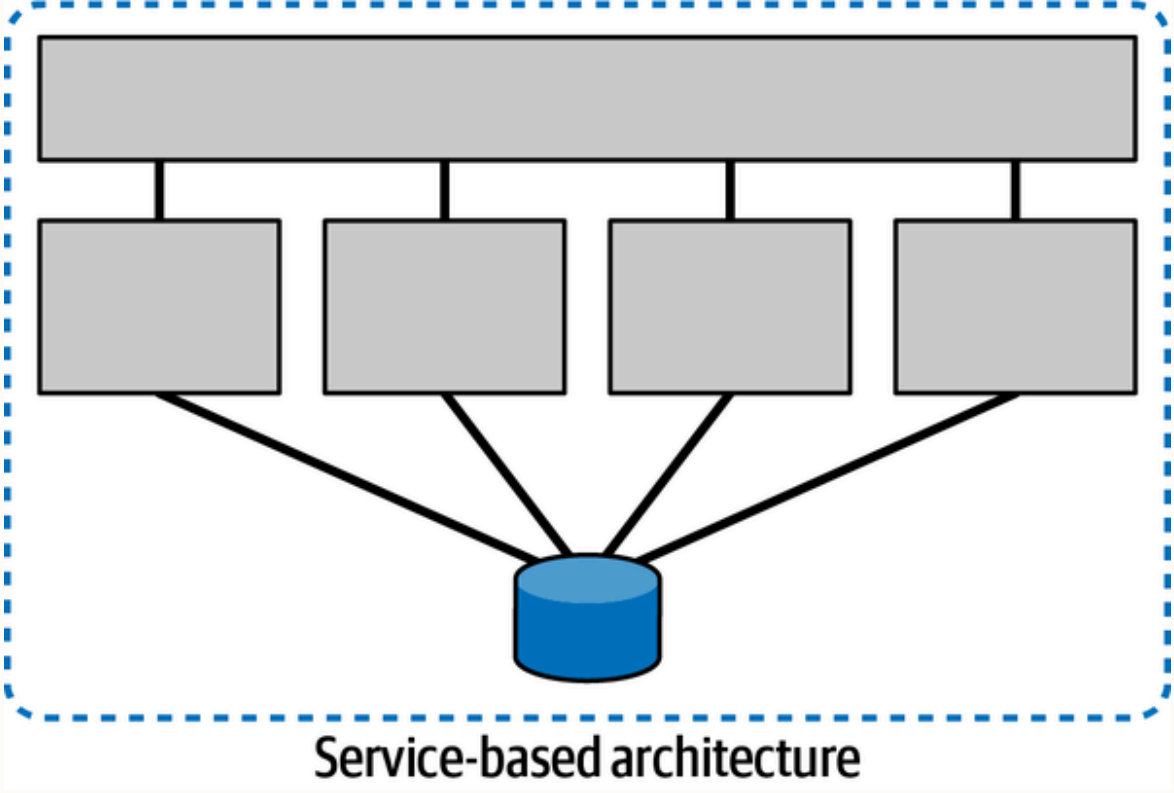


Figure 2-3. Architecture quantum for a service-based architecture

While this individual services model shows the isolation common in microservices, the architecture still utilizes a single relational database, rendering its architecture quantum score to one.

SERVICE-BASED ARCHITECTURE

When we refer to *service-based architecture*, we don't mean a generic architecture based on services, but rather a specific hybrid architecture style that follows a distributed macro-layered structure consisting of a separately deployed user interface, separately deployed remote coarse-grained services, and a monolithic database. This architecture addresses one of the complexities of microservices—separation at the database level. Services in a service-based architecture follow the same principles as microservices (based on domain-driven design's bounded context) but rely on a single relational database because the architects didn't see value in separation (or saw too many negative trade-offs).

Service-based architectures are common targets when restructuring monolithic architectures, allowing for decomposition without disrupting existing database schemas and integration points. We cover decomposition patterns in Chapter 5.

So far, the static coupling measurement of architecture quantum has evaluated all the topologies to *one*. However, distributed architectures create the possibility of multiple quanta but don't necessarily guarantee it. For example, the mediator style of event-driven

architecture will always be evaluated to an single architecture quantum, as illustrated in Figure 2-4.

Even though this style represents a distributed architecture, two coupling points push it toward a single architecture quantum: the database, as common with the previous monolithic architectures, but also the `Request Orchestrator` itself—any holistic coupling point necessary for the architecture to function forms an architecture quantum around it.

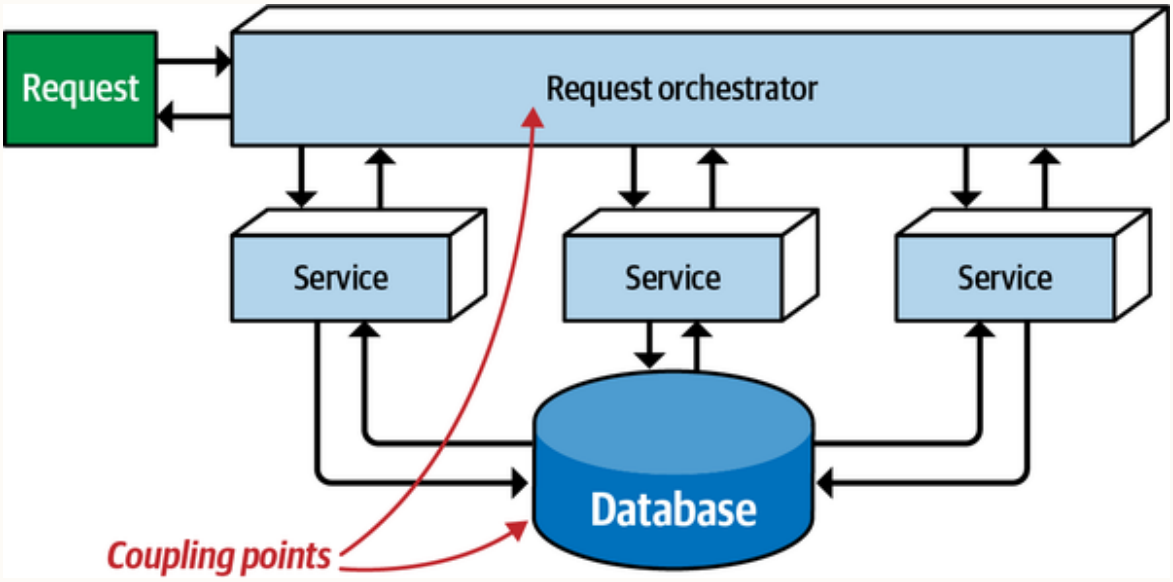


Figure 2-4. A mediated EDA has a single architecture quantum

Broker event-driven architectures (without a central mediator) are less coupled, but that doesn't guarantee complete decoupling. Consider the event-driven architecture illustrated in Figure 2-5.

This broker-style event driven architecture (without a central mediator) is nevertheless a single architecture quantum because all the services utilize a single relational database, which acts as a common coupling point. The question answered by the static analysis for an architecture quantum is, "Is this dependent of the architecture necessary to bootstrap this service?" Even in the case of an event-driven architecture where some of the services don't access the database, if they rely on services that *do* access the database, then they become part of the static coupling of the architecture quantum.

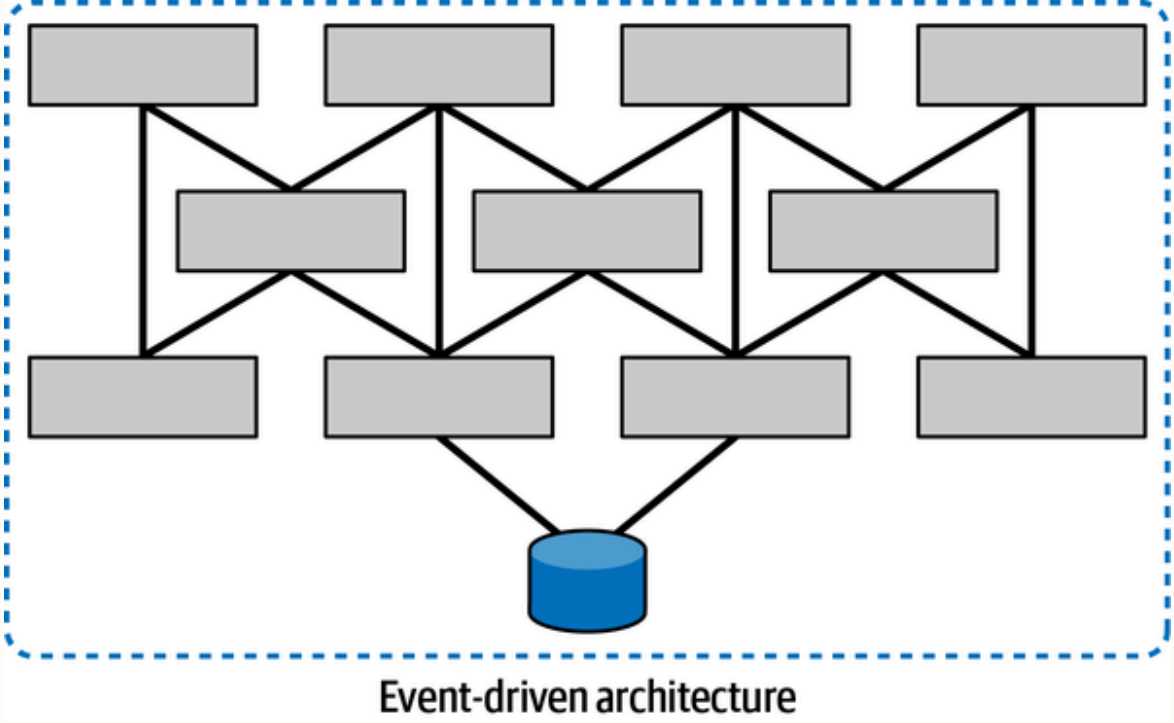


Figure 2-5. Even a distributed architecture such as broker-style event-driven architecture can be a single quantum

However, what about situations in distributed architectures where common coupling points don't exist? Consider the event-driven architecture illustrated in Figure 2-6.

The architects designed this event-driven system with two data stores, and no static dependencies between the sets of services. Note that either architecture quantum can run in a production-like ecosystem. It may not be able to participate in all workflows required by the system, but it runs successfully and operates—sends requests and receives them within the architecture.

The static coupling measure of an architecture quantum assesses the coupling dependencies between architectural and operational components. Thus, the operating system, data store, message broker, container orchestration, and all other operational dependencies form the static coupling points of an architecture quantum, using the strictest possible contracts, operational dependencies (more about the role of contracts in architecture quanta in Chapter 13).

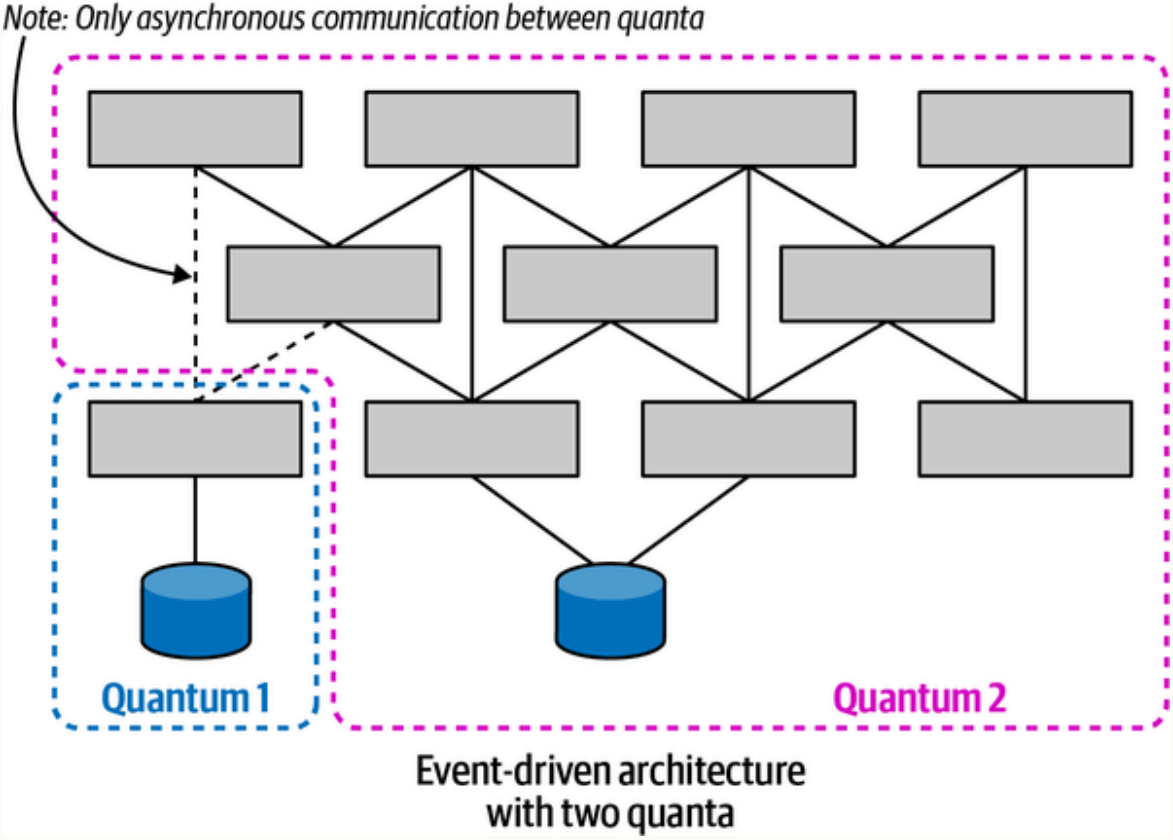


Figure 2-6. An event-driven architecture with multiple quanta

The microservices architecture style features highly decoupled services, including data dependencies. Architects in these architectures favor high degrees of decoupling and take care not to create coupling points between services, allowing each individual service to each form its own quanta, as shown in Figure 2-7.

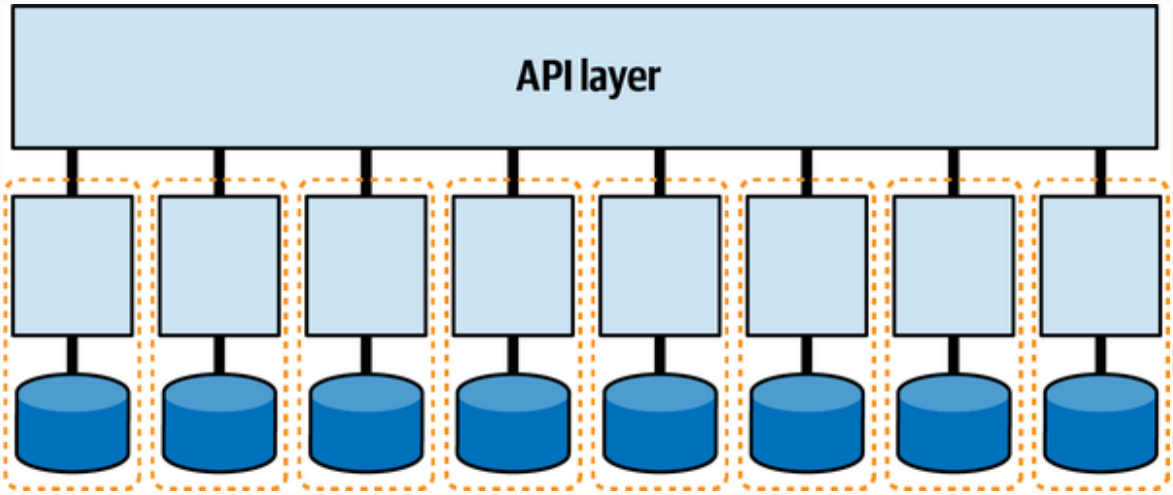


Figure 2-7. Microservices may form their own quanta

Each service (acting as a bounded context) may have its own set of architecture characteristics—one service might have higher levels of scalability or security than another. This granular level of architecture characteristics scoping represents one of the advantages of the microservices architecture style. High degrees of decoupling allow teams working on a service to move as quickly as possible, without worrying about breaking other dependencies.

However, if the system is tightly coupled to a user interface, the architecture forms a single architecture quantum, as illustrated in Figure 2-8.

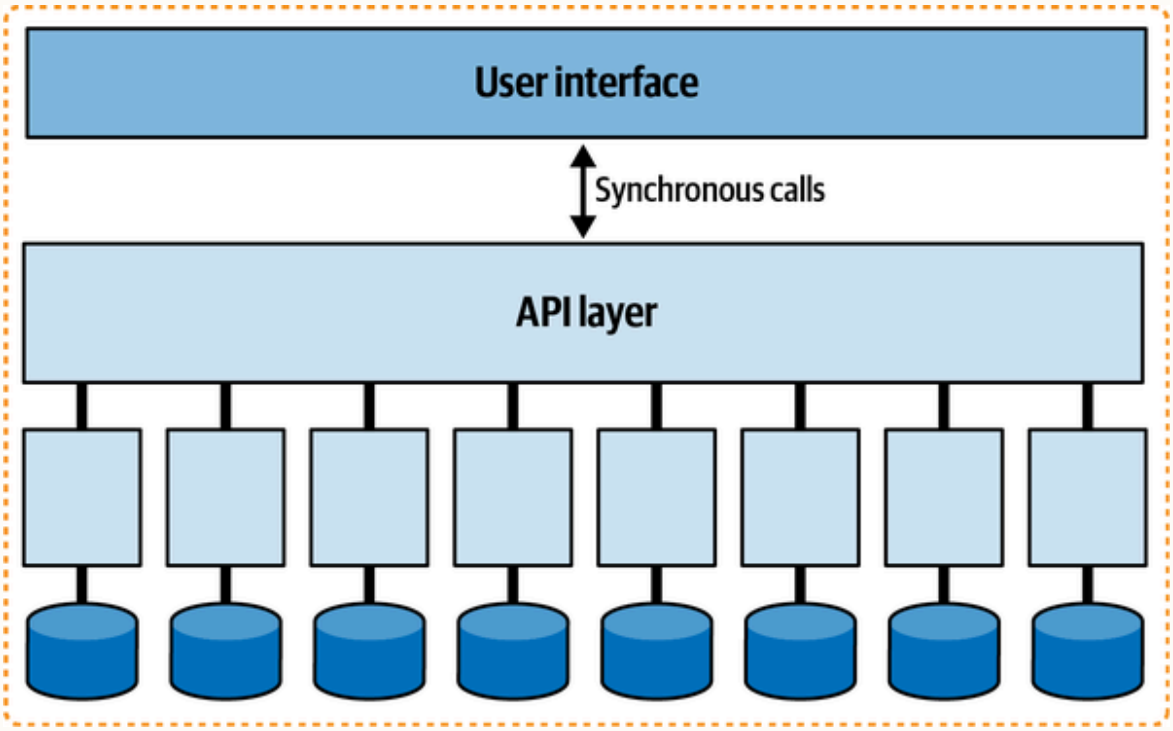


Figure 2-8. A tightly coupled user interface can reduce a microservices architecture quantum to one

User interfaces create coupling points between the front and back end, and most user interfaces won't operate if portions of the backend aren't available.

Additionally, it will be difficult for an architect to design different levels of operational architecture characteristics (performance, scale, elasticity, reliability, and so on) for each service if they all must cooperate together in a single user interface (particularly in the case of synchronous calls, covered in "Dynamic Quantum Coupling").

Architects design user interfaces utilizing asynchronicity that doesn't create coupling between front and back. A trend on many microservices projects is to use a *micro frontend* framework for user interface elements in a microservices architecture. In such an architecture, the user interface elements that interact on behalf of the services are emitted from the services themselves. The user interface surface acts as a canvas where the user interface elements can appear, and also facilitates loosely coupled communication between components, typically using events. Such an architecture is illustrated in Figure 2-9.

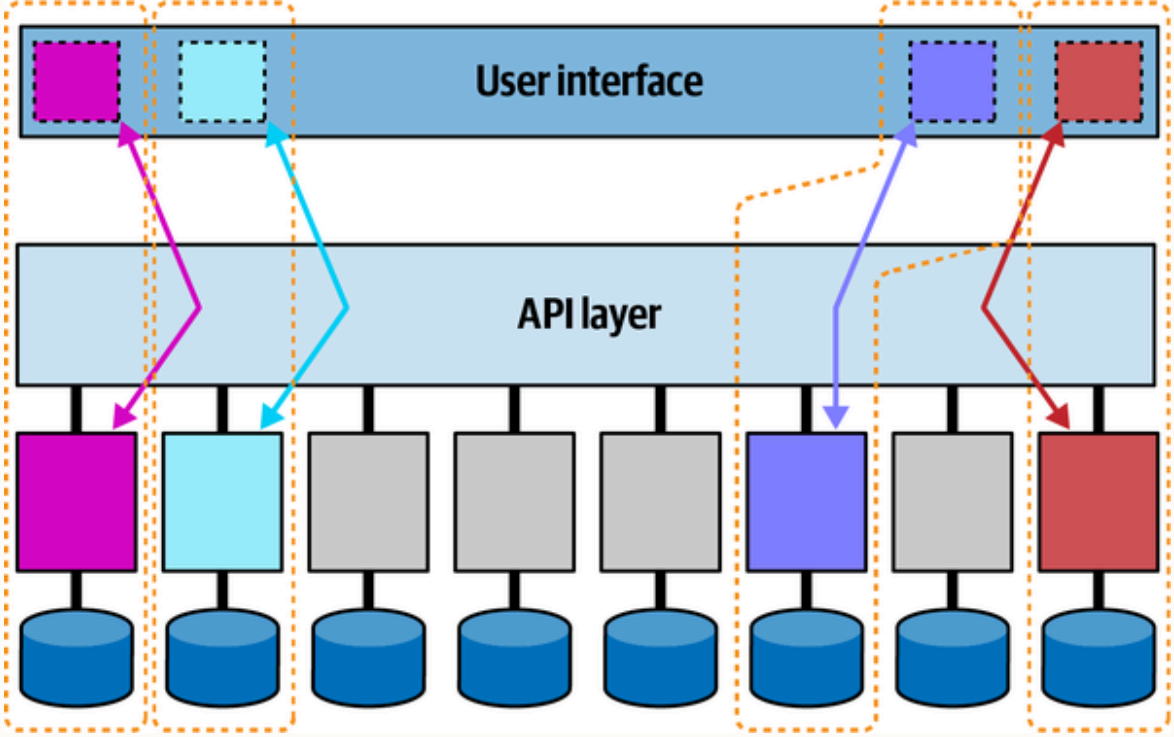


Figure 2-9. In a micro-frontend architecture, each service + user interface component forms an architecture quantum

In this example, the four tinted services along with their corresponding micro-frontends form architecture quanta: each of these services may have different architecture characteristics.

Any coupling point in an architecture can create static coupling points from a quantum standpoint. Consider the impact of a shared database between two systems, as illustrated in Figure 2-10.

The static coupling of a system provides valuable insight, even in complex systems involving integration architecture. Increasingly, a common architect technique for understanding legacy architecture involves creating a static quantum diagram of how things are "wired" together, which helps determine what systems will be impacted by change and offers a way of understanding (and potentially decoupling) the architecture.

Static coupling is only one-half of the forces at play in distributed architectures. The other is dynamic coupling.

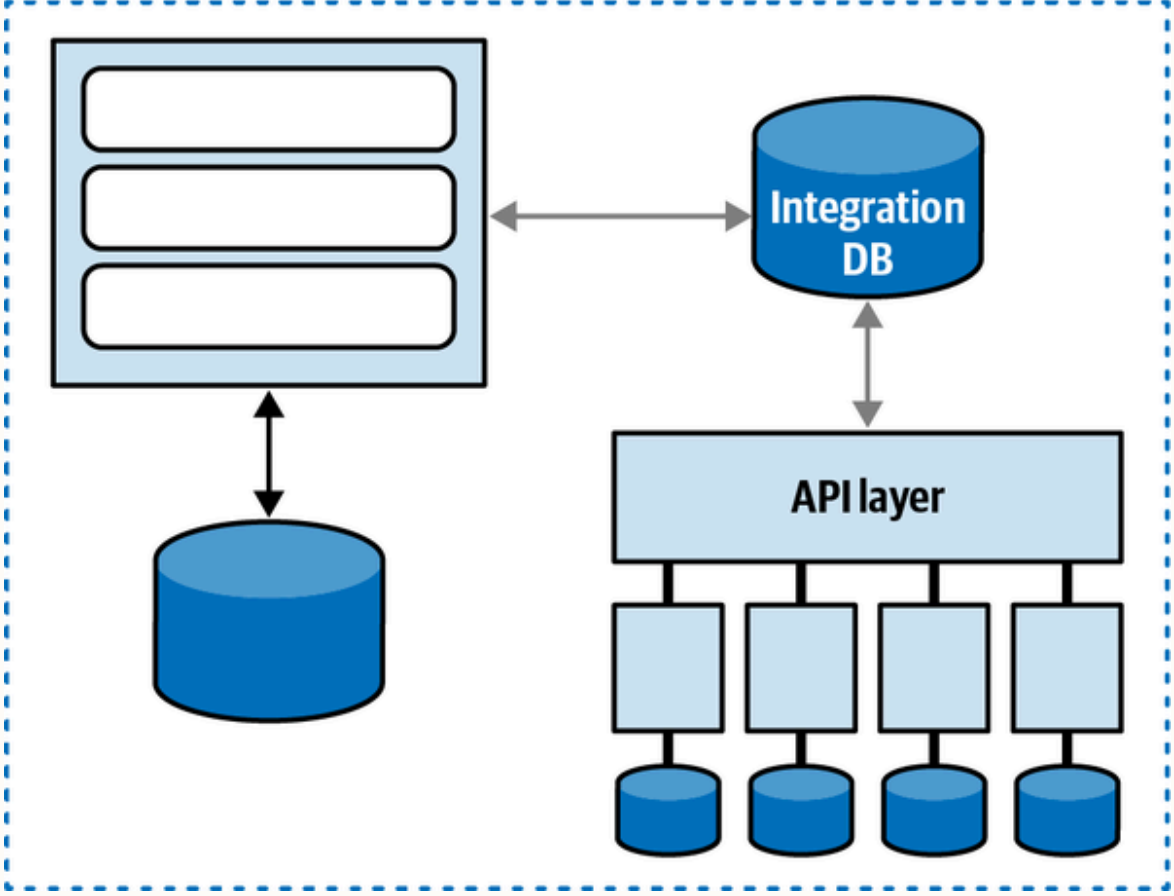


Figure 2-10. A shared database forms a coupling point between two systems, creating a single quantum

Dynamic Quantum Coupling

The last portion of the architecture quantum definition concerns synchronous coupling at runtime—in other words, the behavior of architecture quanta as they interact with one another to form workflows within a distributed architecture.

The nature of *how* services call one another creates difficult trade-off decisions because it represents a multidimensional decision space, influenced by three interlocking forces:

Communication

Refers to the type of connection synchronicity used: synchronous or asynchronous.

Consistency

Describes whether the workflow communication requires atomicity or can utilize eventual consistency.

Coordination

Describes whether the workflow utilizes an orchestrator or whether the services communicate via choreography.

Communication

When two services communicate with each other, one of the fundamental questions for an architect is whether that communication should be synchronous or asynchronous.

Synchronous communication requires the requestor to wait for the response from the receiver, as shown in Figure 2-11.

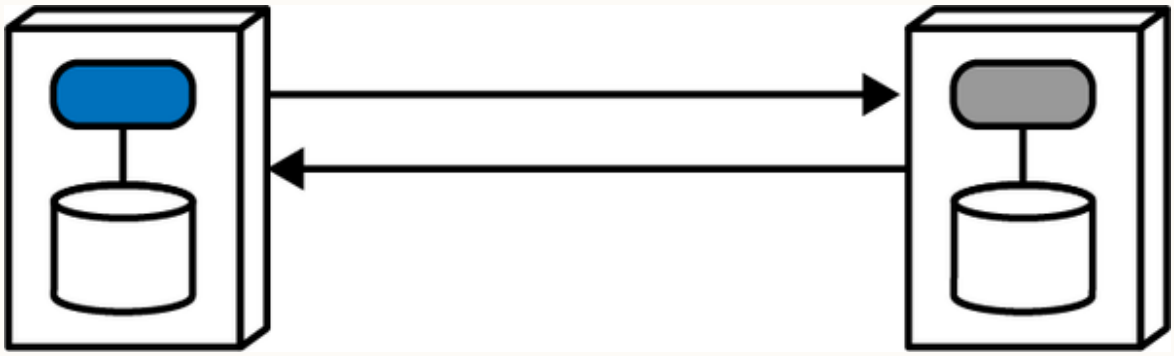


Figure 2-11. A synchronous call waits for a result from the receiver

The calling service makes a call (using one of a number of protocols that support synchronous calls, such as gRPC) and *blocks* (does no further processing) until the receiver returns a value (or status indicating a state change or error condition).

Asynchronous communication occurs between two services when the caller posts a message to the receiver (usually via a mechanism such as a message queue) and, once the caller gets acknowledgment that the message will be processed, it returns to work. If the request required a response value, the receiver can use a reply queue to (asynchronously) notify the caller of the result, which is illustrated in Figure 2-12.

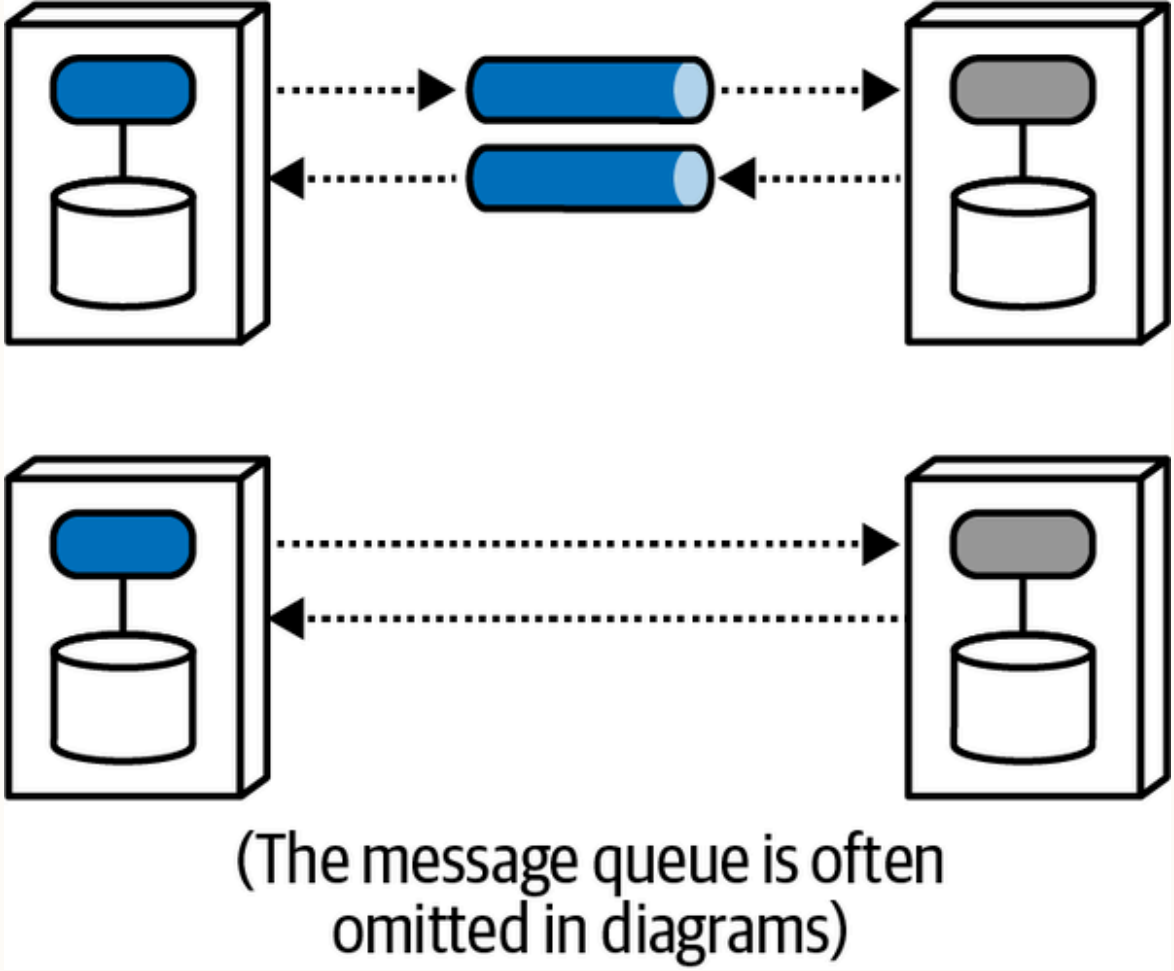


Figure 2-12. Asynchronous communication allows parallel processing

The caller posts a message to a message queue and continues processing until notified by the receiver that the requested information is available via return call. Generally, architects use message queues (illustrated via the gray cylindrical tube in the top diagram in Figure 2-12) to implement asynchronous communication, but queues are common and create noise on diagrams, so many architects leave them off, as shown in the lower diagram. And, of course, architects can implement asynchronous communication without message queues by using a variety of libraries or frameworks. Each diagram variety implies asynchronous messaging; the second provides visual shorthand and less implementation detail.

Architects must consider significant trade-offs when choosing how services will communicate. Decisions around communication affect synchronization, error handling, transactionality, scalability, and performance. The remainder of this book delves into many of these issues.

Consistency

Consistency refers to the strictness of transactional integrity that communication calls must adhere to. Atomic transactions (all-or-nothing transactions requiring consistency *during* the processing of a request) lie on one side of the spectrum, whereas different degrees of eventual consistency lie on the other side.

Transactionality—having several services participate in an all-or-nothing transaction—is one of the most difficult problems to model in distributed architectures, resulting in the general advice to try to avoid cross-service transactions. We discuss consistency and the intersection of data and architecture in Chapters 6, 9, 10, and 12.

Coordination

Coordination refers to how much coordination the workflow modeled by the communication requires. The two common generic patterns for microservices are orchestration and choreography, which we describe in Chapter 11. Simple workflows—a single service replying to a request—don't require special consideration from this dimension. However, as the complexity of the workflow grows, the greater the need for coordination.

These three factors—communication, consistency, and coordination—all inform the important decision an architect must make. Critically, however, architects cannot make these choices in isolation; each option has a gravitation effect on the others. For example, transactionality is easier in synchronous architectures with mediation, whereas higher levels of scale are possible with eventually consistent asynchronous choreographed systems.

Thinking about these forces as related to each other forms a three-dimensional space, illustrated in Figure 2-13.

Each force in play during service communication appears as a dimension. For a particular decision, an architect could graph the position in space representing the strength of these forces.

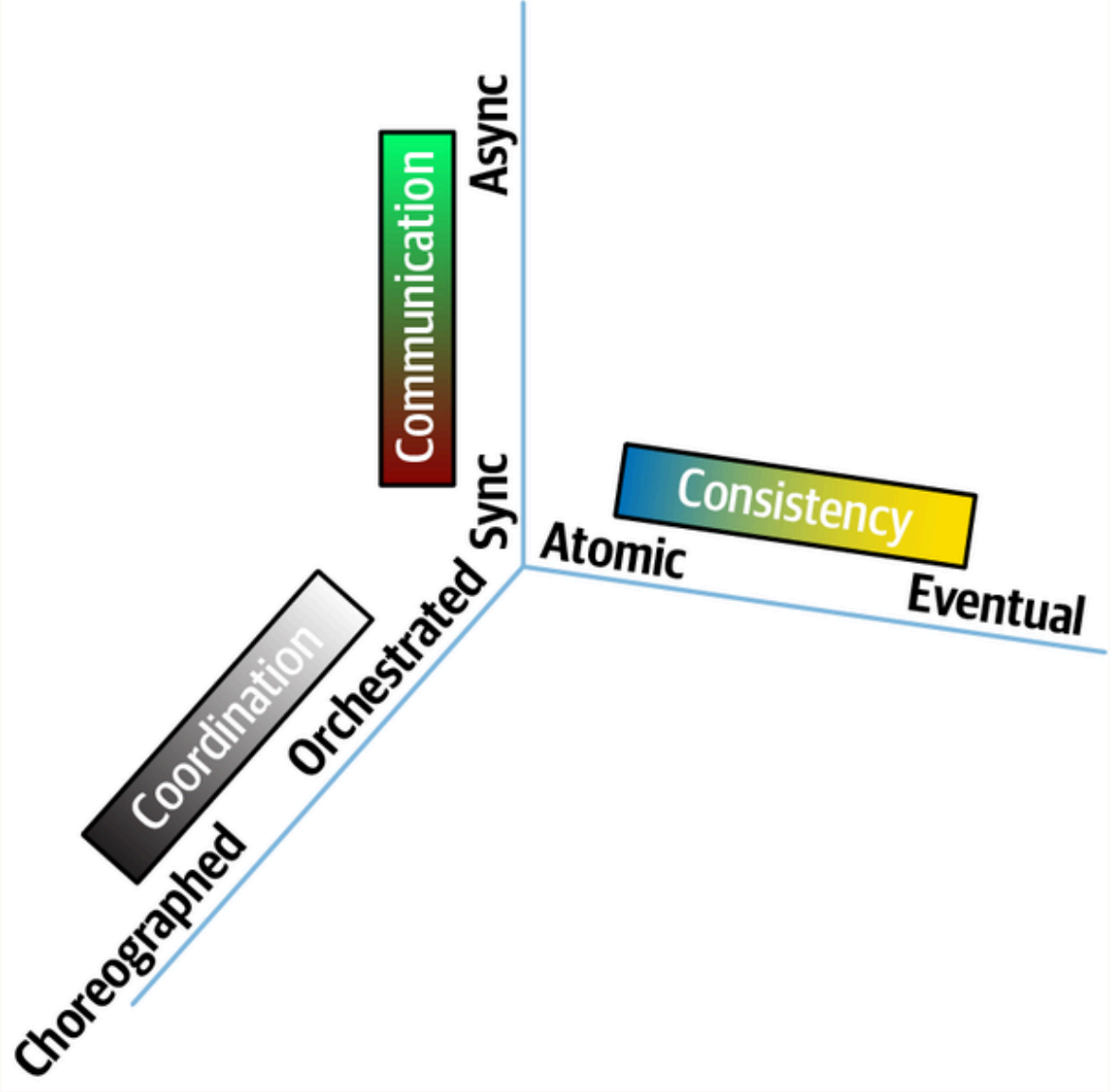


Figure 2-13. The dimensions of dynamic quantum coupling

When an architect can build a clear understanding of forces at play within a given situation, it creates criteria for trade-off analysis. In the case of dynamic coupling, Table 2-1 shows a framework for identifying fundamental pattern names based on the eight possible combinations.

Table 2-1. The matrix of dimensional intersections for distributed architectures

Pattern name	Communication	Consistency	Coordination	Coupling
Epic Saga(sao)	synchronous	atomic	orchestrated	very high
Phone Tag Saga(sac)	synchronous	atomic	chore-ographed	high
Fairy Tale Saga(seo)	synchronous	eventual	orchestrated	high
Time Travel Saga(sec)	synchronous	eventual	chore-ographed	medium
Fantasy Fiction Saga(aao)	asynchronous	atomic	orchestrated	high
Horror Story(aac)	asynchronous	atomic	chore-ographed	medium
Parallel Saga(aeo)	asynchronous	eventual	orchestrated	low
Anthology Saga(aec)	asynchronous	eventual	chore-ographed	very low

To fully understand this matrix, we must first investigate each of the dimensions individually. Therefore, the following chapters help you build context to understand the individual trade-offs for communication, consistency, and coordination, then entangle them back together in Chapter 12.

In the remaining chapters in Part I, we focus on static coupling and understanding the various dimensions at play in distributed architectures, including data ownership, transactionality, and service granularity. In Part II, "Putting Things Back Together", we focus on dynamic coupling and understanding communication patterns in microservices.

Sysops Squad Saga: Understanding Quanta

Tuesday, November 23, 14:32



Austen came to Addison's office wearing an uncharacteristic cross expression. "Hey, Addison, can I bother you for a minute?"

"Sure, what's up?"

"I've been reading about this architecture quantum stuff, and I just...don't...get...it!"

Addison laughed, "I know what you mean. I struggled with it when it was purely abstract, but when you ground it in practical things, it turns out to be a useful set of perspectives."

"What do you mean?"

"Well," said Addison, "the architecture quantum basically defines a DDD bounded context in architectural terms."

"Why not just use bounded context, then?" asked Austen.

"Bounded context has a specific definition in DDD, and overloading it with stuff about architecture just makes people constantly have to differentiate. They are similar, but not the same thing. The first part about *functional cohesion* and *independent deployment* certainly matches a service based on bounded context. But the architecture quantum definition goes further by identifying types of coupling—that's where the *static* and *dynamic* stuff comes in."

"What is that all about? Isn't *coupling* just *coupling*? Why make the distinction?"

"It turns out that a bunch of different concerns revolve around the different types," said Addison. "Let's take the *static* one first, which I like to think of as how things are wired together. Another way to think about it: consider one of the services we're building in our target architecture. What is all the wiring required to bootstrap that service?"

"Well, it's written in Java, using a Postgres database, and running in Docker—that's it, right?"

"You're missing a lot." said Addison. "What if you had to build that service from scratch, assuming we had nothing in place? It's Java, but also using SpringBoot and, what, about 15 or 20 different frameworks and libraries?"

"That's right, we can look in the Maven POM file to figure out all those dependencies. What else?"

"The idea behind *static quantum coupling* is the wiring required to function. We're using events to communicate between services—what about the event broker?"

"But isn't that the dynamic part?"

"Not the *presence* of the broker. If the service (or, more broadly, architecture quantum) I want to bootstrap utilizes a message broker to function, the broker must be present. When the service *calls* another service via the broker, we get into the dynamic side."

"OK, that makes sense," said Austen. "If I think about what it would take to bootstrap it from scratch, that's the static quantum coupling."

"That's right. And just that information is super useful. We recently built a diagram of the static quantum coupling for each of our services defensively."

Austen laughed. "Defensively? What do you..."

"We were performing a reliability analysis to determine if I change this *thing*, what might break, where *thing* could be anything in our architecture or operations. They're trying to do risk mitigation—if we change a service, they want to know what must be tested."

"I see—that's the static quantum coupling. I can see how that's a useful view. It also shows how teams might impact one another. That seems really useful. Is there a tool we can download that figures that out for us?"

"Wouldn't that be nice!" laughed Addison. "Unfortunately, no one with our unique mix of architecture has built and open sourced exactly the tool we want. However, some of the platform team is working on a tool to automate it, necessarily customized to our architecture. They're using the container manifests, POM files, NPM dependencies, and other dependency tools to build and maintain a list of build dependencies. We have also instituted observability for all our services, so we now have consistent log files about what systems call each other, when, and how often. They're using that to build a call graph to see how things are connected."

"OK, so static coupling is how things are wired together. What about *dynamic* coupling?"

"Dynamic coupling concerns how quanta *communicate* with each other, particularly synchronous versus asynchronous calls and their impact on operational architecture character-

istics—things like performance, scale, elasticity, reliability, and so on. Consider *elasticity* for a moment—remember the difference between scalability and elasticity?"

Austen smirked. "I didn't know there was going to be a test. Let's see...*scalability* is the ability to support a large number of concurrent users; *elasticity* is the ability to support a burst of user requests in a short time frame."

"Correct! Gold star for you. OK, let's think about *elasticity*. Suppose in our future state architecture we have two services like Ticketing and Assignment, and the two types of calls. We carefully designed our services to be highly statically decoupled from one another, so that they can be independently elastic. That's the other side effect of static coupling, by the way—it identifies the scope of things like operational architecture characteristics. Let's say that Ticketing is operating at 10 times the elastic scale of Assignment, and we need to make a call between them. If we make a synchronous call, the whole workflow will bog down, as the caller waits for the slower service to process and return. If on the other hand we make an asynchronous call, using the message queue as a buffer, we can allow the two services to execute operationally independently, allowing the caller to add messages to the queue and continue working, receiving notification when the workflow is complete."

"Oh, I see, I see! The architecture quantum defines the scope of architecture characteristics—it's obvious how the static coupling can affect that. But I see now that, depending on the type of call you make, you might temporarily couple two services together."

"That's right," said Addison. "The architecture quanta can entangle one another temporarily, during the course of a call, if the nature of the call ties things like performance, responsiveness, scale, and a bunch of others."

"OK, I think I understand what an architecture quantum is, and how the coupling definitions work. But I'm never going to get that quantum/quanta thing straight!"

"Same for datum/data, but no one ever uses datum!" laughed Addison. "You'll see a lot more of the impact of dynamic coupling on workflows and transactional sagas as you keep digging into our architecture."

"I can't wait!"

Previous chapter

< I. Pulling Things Apart

Next chapter

3. Architectural Modularity >